SIMPLY FPU

by **Raymond Filiatreault**

Chap. 8
Arithmetic instructions - with REAL numbers

The FPU instructions covered in this chapter perform arithmetic operations on floating point values. The value in the FPU's TOP data register ST(0) must always be one of the operands, whether explicitly or implicitly.

The arithmetic instructions covered in this document are (in alphabetical order):

FABS	ABSolute value of ST(0)
FADD	ADD two floating point values
FADDP	ADD two floating point values and Pop ST(0)
FCHS	CHange the Sign of ST(0)
FDIV	DIVide two floating point values
FDIVP	DIVide two floating point values and Pop ST(0)
FDIVR	DIVide in Reverse two floating point values
FDIVRP	DIVide in Reverse two floating point values and Pop ST(0)
FMUL	MULtiply two floating point values
FMULP	MULtiply two floating point values and Pop ST(0)
FRNDINT	RouND ST(0) to an INTeger
FSQRT	Square RooT of ST(0)
FSUB	SUBtract two floating point values
FSUBP	SUBtract two floating point values and Pop ST(0)
FSUBR	SUBtract in Reverse two floating point values
FSUBRP	SUBtract in Reverse two floating point values and Pop ST(0)

FADD (Add two floating point values)

```
Syntax:    fadd Src
           fadd Dest,Src
```

Note: FADD without operands can also be used with the MASM assembler but such an instruction is coded as FADDP ST(1),ST(0).

```
Exception flags: Stack Fault, Invalid operation, Denormalized value,
                 Overflow, Underflow, Precision
```

This instruction performs a signed addition of the floating point values from the source (*Src*) and the destination (*Dest*) and overwrites the content of the destination data register with the result. If only the source is specified, the TOP data register ST(0) is the implied destination and the source must be the memory address of a REAL4 or REAL8 value (see Chap.2 for addressing modes of real numbers). If both the source

and destination are specified, they must both be FPU data registers and one of them must be the TOP data register ST(0). *(The FIADD instruction must be used to add an integer located in memory to the value in ST(0)).*

> Note that a REAL10 in memory cannot be added directly to ST(0). If such an addition becomes necessary, the REAL10 value must first be loaded to the FPU.

An **I**nvalid operation exception is detected if a specified data register is empty, or if either of the two values is a NAN, setting the related flag in the Status Word. The destination register would be overwritten with the INDEFINITE value. (Values of INFINITY will be treated as valid numbers and yield an INFINITY result without any exception being detected, unless both values are INFINITY of opposite sign.)

A **S**tack **F**ault exception is also detected if a data register specified as an operand is empty, setting the related flag in the Status Word.

A **D**enormal exception is detected when either of the two operands is a denormalized number, setting the related flag in the Status Word. The addition would still yield a valid result.

A **P**recision exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An **O**verflow or **U**nderflow exception will be detected if the result exceeds the range limits of REAL10 numbers, setting the related flag in the Status Word and the value of INFINITY will overwrite the content of the destination register in case of overflow.

If a REAL4 or REAL8 value in memory is specified as the source, it is first converted to the REAL10 format before the addition. If that value is not an exact number in binary, the precision of the result will only be equivalent to the precision of the source.

Examples of use:

```
fadd real8_var       ;add the value of the real8_var variable to ST(0)
fadd dword ptr [eax] ;add the REAL4 value pointed to by EAX to ST(0)
fadd st,st           ;double the value of ST(0)
fadd st,st(3)        ;add the values of ST(0) and ST(3), result in ST(0)
fadd st(3),st        ;add the values of ST(0) and ST(3), result in ST(3)
```

FADDP (Add two floating point values and Pop ST(0))

```
Syntax:    faddp st(i),st

Exception flags: Stack Fault, Invalid operation, Denormalized value,
                 Overflow, Underflow, Precision
```

This instruction performs a signed addition of the content from the ST(i) and ST(0) registers. The result then overwrites the content of the ST(i) register and the TOP data register is popped.

An **I**nvalid operation exception is detected if a specified data register is empty, or if its value is a NAN, setting the related flag in the Status Word. The destination register would be overwritten with the INDEFINITE value. (Values of INFINITY will be treated as valid numbers and yield an INFINITY result without any exception being detected, unless both values are INFINITY of opposite sign.)

A **S**tack **F**ault exception is also detected if a data register specified as an operand is empty, setting the related flag in the Status Word.

A **D**enormal exception is detected when either of the two operands is a denormalized number, setting the related flag in the Status Word. The addition would still yield a valid result.

A **P**recision exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An **O**verflow or **U**nderflow exception will be detected if the result exceeds the range limits of REAL10 numbers, setting the related flag in the Status Word and the value of INFINITY will overwrite the content of the destination register in case of overflow

This instruction is used when the value in ST(0) would no longer be needed for further computation. Examples of use could be as follows.

Example 1: Adding to ST(0) a temporary REAL10 value located in memory which needs to be loaded to the FPU for such an operation.

```
fld   real10_var  ;load the temporary value
                  ;-> ST(0)=real10_var, ST(1)=previous ST(0)
faddp st(1),st    ;add the previous ST(0) and the real10_var and discard it
                  ;-> ST(0)=(previous ST(0) + real10_var)
```

Note that MASM would also accept **fadd** (no operand) instead of the **faddp st(1),st** instruction.

Example 2: Computing a sum of squares from an array of REAL8 values

```
lea   edx,real8_array ;use EDX as a pointer to the array
mov   ecx,array_items ;use ECX as counter for the number of items to process
fldz                  ;initialize a register to accumulate the squared values
                      ;-> ST(0)=accumulator=0
@@:
fld   qword ptr [edx] ;load the next REAL8 value
                      ;-> ST(0)=REAL value, ST(1)=accumulator
fmul  st,st           ;square it
                      ;-> ST(0)=REAL value squared, ST(1)=accumulator
faddp st(1),st        ;add to the accumulator and discard the squared value
                      ;-> ST(0)=accumulator+squared REAL value
add   edx,8           ;adjust the pointer to the next REAL8 item
dec   ecx             ;decrement the counter
jnz   @B              ;continue until all items are processed

                      ;-> ST(0)=the sum of squares
```

FMUL (Multiply two floating point values)

```
Syntax:    fmul Src
           fmul Dest,Src
```

Note: FMUL without operands can also be used with the MASM assembler but such an instruction is coded as FMULP ST(1),ST(0).

```
Exception flags: Stack Fault, Invalid operation, Denormalized value,
                 Overflow, Underflow, Precision
```

This instruction performs a signed multiplication of the floating point values from the source (*Src*) and the destination (*Dest*) and overwrites the content of the destination data register with the result; the value of the source remains unchanged. If only the source is specified, the TOP data register ST(0) is the implied destination and the source must be the memory address of a REAL4 or REAL8 value (see Chap.2 for addressing modes of real numbers). If both the source and destination are specified, they must both be FPU data registers and one of them must be the TOP data register ST(0). *(The FIMUL instruction must be used to multiply an integer located in memory with the value in ST(0)).*

Note that a REAL10 in memory cannot be multiplied directly with ST(0). If such an multiplication becomes necessary, the REAL10 value must first be loaded to the FPU.

An **I**nvalid operation exception is detected if a specified data register is empty, or if either of the two values is a NAN, setting the related flag in the Status Word. The destination register would be overwritten with the INDEFINITE value. (Values of INFINITY will be treated as valid numbers and yield an INFINITY result without any exception being detected, unless the other operand has a value of zero.)

A **S**tack **F**ault exception is also detected if a data register specified as an operand is empty, setting the related flag in the Status Word.

A **D**enormal exception is detected when either of the two operands is a denormalized number, setting the related flag in the Status Word. The multiplication would still yield a valid result.

A **P**recision exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An **O**verflow or **U**nderflow exception will be detected if the result exceeds the range limits of REAL10 numbers, setting the related flag in the Status Word and the value of INFINITY will overwrite the content of the destination register in case of overflow.

If a REAL4 or REAL8 value in memory is specified as the source, it is first converted to the REAL10 format before the addition. If that value is not an exact number in binary, the precision of the result will only be equivalent to the precision of the source.

Examples of use:

```
fmul real8_var          ;multiply ST(0) by the real8_var variable
fmul dword ptr [eax]    ;multiply ST(0) by the REAL4 pointed to by EAX
fmul qword ptr [esi+8]  ;multiply ST(0) by the REAL8 pointed to by ESI+8
fmul st,st              ;square the value of ST(0)
fmul st,st(3)           ;multiply ST(3) and ST(0), result in ST(0)
fmul st(3),st           ;multiply ST(3) and ST(0), result in ST(3)
```

FMULP (Multiply two floating point values and Pop ST(0))

```
Syntax:    fmulp st(i),st

Exception flags: Stack Fault, Invalid operation, Denormalized value,
                 Overflow, Underflow, Precision
```

This instruction performs a signed multiplication of the contents from the ST(i) and ST(0) data registers. The result then overwrites the content of the ST(i) register and the TOP data register is popped.

An **I**nvalid operation exception is detected if a specified data register is empty, or if its value is a NAN, setting the related flag in the Status Word. The destination register would be overwritten with the INDEFINITE value. (Values of INFINITY will be treated as valid numbers and yield an INFINITY result without any exception being detected, unless the other operand has a value of zero.)

A **S**tack **F**ault exception is also detected if a data register specified as an operand is empty, setting the related flag in the Status Word.

A **D**enormal exception is detected when either of the two operands is a denormalized number, setting the related flag in the Status Word. The multiplication would still yield a valid result.

A **P**recision exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An **O**verflow or **U**nderflow exception will be detected if the result exceeds the range limits of REAL10 numbers, setting the related flag in the Status Word and the value of INFINITY will overwrite the content of the destination register in case of overflow.

This instruction is used when the value in ST(0) would no longer be needed for further computation. An example would be when the value of a data register needs to be multiplied by a temporary REAL10 located in memory (or by one of the constants hard-coded on the FPU). The following code assumes that the value in ST(2) needs to be multiplied by the value of PI.

```
fldpi           ;load the hard-coded PI constant with 64 bits of precision
                ;-> ST(0)=PI, ST(3)=previous ST(2)
```

```
fmulp st(3),st ;multiply, overwrite ST(3) with result, then discard PI
               ;-> ST(2)=previous ST(2) multiplied by PI
```

Note that MASM would also accept **fmul** (no operand) instead of an **fmulp st(1),st** instruction.

FSUB (Subtract two floating point values)

```
Syntax:    fsub Src
           fsub Dest,Src
```

Note: FSUB without operands can also be used with the MASM assembler but such an instruction is coded as FSUBP ST(1),ST(0).

```
Exception flags: Stack Fault, Invalid operation, Denormalized value,
                 Overflow, Underflow, Precision
```

This instruction performs a signed subtraction of the floating point value of the source (*Src*) operand from the value of the destination (*Dest*) register operand and overwrites the content of the destination data register with the result; the value of the source remains unchanged. If only the source is specified, the TOP data register ST(0) is the implied destination and the source must be the memory address of a REAL4 or REAL8 value (see Chap.2 for addressing modes of real numbers). If both the source and destination are specified, they must both be FPU data registers and one of them must be the TOP data register ST(0). *(The FISUB instruction must be used to subtract an integer located in memory from the value in ST(0)).*

Note that a REAL10 in memory cannot be subtracted directly from ST(0). If such a subtraction becomes necessary, the REAL10 value must first be loaded to the FPU.

An **I**nvalid operation exception is detected if a specified data register is empty, or if either of the two values is a NAN, setting the related flag in the Status Word. The destination register would be overwritten with the INDEFINITE value. (Values of INFINITY will be treated as valid numbers and yield a signed INFINITY result without any exception being detected, unless both values are INFINITY of the same sign.)

A **S**tack **F**ault exception is also detected if a data register specified as an operand is empty, setting the related flag in the Status Word.

A **D**enormal exception is detected when either of the two operands is a denormalized number, setting the related flag in the Status Word. The subtraction would still yield a valid result.

A **P**recision exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An **O**verflow or **U**nderflow exception will be detected if the result exceeds the range limits of REAL10 numbers, setting the related flag in the Status Word and the value of INFINITY will overwrite the content of the destination register in case of overflow.

If a REAL4 or REAL8 value in memory is specified as the source, it is first converted to the REAL10 format before the subtraction. If that value is not an exact number in binary, the precision of the result will only be equivalent to the precision of the source.

Examples of use:

```
fsub real8_var      ;subtract the value of the real8_var variable from ST(0)
fsub dword ptr[eax] ;subtract the REAL4 value pointed to by EAX from ST(0)
fsub st,st          ;would result in a value of 0.0 in ST(0)
fsub st,st(3)       ;subtract the value of ST(3) from ST(0), result in ST(0)
fsub st(3),st       ;subtract the value of ST(0) from ST(3), result in ST(3)
```

FSUBP (Subtract two floating point values and pop ST(0))

```
Syntax:    fsubp st(i),st

Exception flags: Stack Fault, Invalid operation, Denormalized value,
                 Overflow, Underflow, Precision
```

This instruction performs a signed subtraction of the content of the ST(0) data register from the content of the ST(i) data register. The result then overwrites the content of the ST(i) register and the TOP data register is popped.

An **I**nvalid operation exception is detected if a specified data register is empty, or if either of the two values is a NAN, setting the related flag in the Status Word. The destination register would be overwritten with the INDEFINITE value. (Values of INFINITY will be treated as valid numbers and yield a signed INFINITY result without any exception being detected, unless both values are INFINITY of the same sign.)

A **S**tack **F**ault exception is also detected if a data register specified as an operand is empty, setting the related flag in the Status Word.

A **D**enormal exception is detected when either of the two operands is a denormalized number, setting the related flag in the Status Word. The subtraction would still yield a valid result.

A **P**recision exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An **O**verflow or **U**nderflow exception will be detected if the result exceeds the range limits of REAL10 numbers, setting the related flag in the Status Word and the value of INFINITY will overwrite the content of the destination register in case of overflow.

This instruction is used when the value in ST(0) would no longer be needed for further computation. An example of use could be when subtracting from ST(0) a temporary REAL10 value located in memory which needs to be loaded to the FPU for such an operation.

```
fld   real10_var ;load the temporary value
                 ;-> ST(0)=real10_var, ST(1)=original ST(0)
fsubp st(1),st   ;subtract real10_var from the original ST(0) and discard it,
                 ;-> ST(0)=(original ST(0) - real10_var)
```

Note that MASM would also accept **fsub** (no operand) instead of the **fsubp st(1),st** instruction.

FSUBR (Subtract in reverse two floating point values)

```
Syntax:    fsubr Src
           fsubr Dest,Src
```

Note: FSUBR without operands can also be used with the MASM assembler but such an instruction is coded as FSUBRP ST(1),ST(0).

```
Exception flags: Stack Fault, Invalid operation, Denormalized value,
                 Overflow, Underflow, Precision
```

This instruction performs a signed subtraction of the content of the destination (*Dest*) register operand from the content of the source (*Src*) operand and overwrites the content of the destination data register with the result; the value of the source remains unchanged. If only the source is specified, the TOP data register ST(0) is the implied destination and the source must be the memory address of a REAL4 or REAL8 value (see Chap.2 for addressing modes of real numbers). If both the source and destination are specified, they must both be FPU data registers and one of them must be the TOP data register ST(0). *(The FISUBR instruction must be used to subtract ST(0) from the value of an integer located in memory).*

Note that a REAL10 located in memory cannot be used with this instruction. If such an subtraction becomes necessary, the REAL10 value must first be loaded to the FPU.

An **I**nvalid operation exception is detected if a specified data register is empty, or if either of the two values is a NAN, setting the related flag in the Status Word. The destination register would be overwritten with the INDEFINITE value. (Values of INFINITY will be treated as valid numbers and yield a signed INFINITY result without any exception being detected, unless both values are INFINITY of the same sign.)

A **S**tack **F**ault exception is also detected if a data register specified as an operand is empty, setting the related flag in the Status Word.

A **D**enormal exception is detected when either of the two operands is a denormalized number, setting the related flag in the Status Word. The subtraction would still yield a valid result.

A **P**recision exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An **O**verflow or **U**nderflow exception will be detected if the result exceeds the range limits of REAL10 numbers, setting the related flag in the Status Word and the value of INFINITY will overwrite the content of the destination register in case of overflow.

If a REAL4 or REAL8 value in memory is specified as the source, it is first converted to the REAL10 format before the subtraction. If that value is not an exact number in binary, the precision of the result will only be equivalent to the precision of the source.

Examples of use:

```
fsubr real8_var      ;subtract ST(0) from the value of the real8_var variable
                     ;and store the result in ST(0)
fsubr dword ptr[eax] ;subtract ST(0) from the REAL4 value pointed to by EAX
                     ;and store the result in ST(0)
fsubr st,st          ;would result in a value of 0.0 in ST(0)
fsubr st,st(3)       ;subtract the value of ST(0) from ST(3), result in ST(0)
                     ;ST(3) remains unchanged
fsubr st(3),st       ;subtract the value of ST(3) from ST(0), result in ST(3)
                     ;ST(0) remains unchanged
```

FSUBRP (Subtract in reverse two floating point values and pop ST(0))

```
Syntax:    fsubrp st(i),st

Exception flags: Stack Fault, Invalid operation, Denormalized value,
                 Overflow, Underflow, Precision
```

This instruction performs a signed subtraction of the content of the ST(i) data register from the content of ST(0) and overwrites the content of the ST(i) register with the result. The TOP data register is then popped.

An **I**nvalid operation exception is detected if a specified data register is empty, or if either of the two values is a NAN, setting the related flag in the Status Word. The destination register would be overwritten with the INDEFINITE value. (Values of INFINITY will be treated as valid numbers and yield a signed INFINITY result without any exception being detected, unless both values are INFINITY of the same sign.)

A **S**tack **F**ault exception is also detected if a data register specified as an operand is empty, setting the related flag in the Status Word.

A **D**enormal exception is detected when either of the two operands is a denormalized number, setting the related flag in the Status Word. The subtraction would still yield a valid result.

A **P**recision exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An **O**verflow or **U**nderflow exception will be detected if the result exceeds the range limits of REAL10 numbers, setting the related flag in the Status Word and the value of INFINITY will overwrite the content of the destination register in case of overflow.

This instruction is used when the value in ST(0) would no longer be needed for further computation. An example of use could be when a subtraction of ST(0) from a temporary REAL10 value located in memory is needed, which requires loading it to the FPU for such an operation.

```
fld   real10_var ;load the temporary value
                 ;-> ST(0)=real10_var, ST(1)=original ST(0)
fsubrp st(1),st  ;subtract the original ST(0) from real10_var and discard it
                 ;-> ST(0)=(temporary value - original ST(0))
```

Note that MASM would also accept **fsubr** (no operand) instead of the **fsubrp st(1),st** instruction.

FDIV (Divide two floating point values)

```
Syntax:    fdiv Src
           fdiv Dest,Src
```

Note: FDIV without operands can also be used with the MASM assembler but such an instruction is coded as FDIVP ST(1),ST(0).

```
Exception flags: Stack Fault, Invalid operation, Denormalized value,
                 Overflow, Underflow, Precision, Zero divide
```

This instruction performs a signed division of the content of the destination (*Dest*) register operand by the content of the source (*Src*) operand and overwrites the content of the destination data register with the result; the value of the source remains unchanged. If only the source is specified, the TOP data register ST(0) is the implied destination and the source must be the memory address of a REAL4 or REAL8 value (see Chap.2 for addressing modes of real numbers). If both the source and destination are specified, they must both be FPU data registers and one of them must be the TOP data register ST(0). *(The FIDIV instruction must be used to divide the value in ST(0) by an integer located in memory).*

Note that ST(0) cannot be divided directly by a REAL10 in memory. If such a division becomes necessary, the REAL10 value must first be loaded to the FPU.

An **I**nvalid operation exception is detected if a specified data register is empty, if either of the two values is a NAN, if the content of the destination register is INFINITY, or if both values are zero, setting the related flag in the Status Word. The destination register would be overwritten with the INDEFINITE value. (A value of INFINITY will be treated as a valid number for the source and yield a signed zero result without any exception being detected, unless both values are INFINITY regardless of sign).

A **S**tack **F**ault exception is also detected if a data register specified as an operand is empty, setting the related flag in the Status Word.

A **D**enormal exception is detected when either of the two operands is a denormalized number, setting the related flag in the Status Word. The division would still yield a valid result.

A **P**recision exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An **O**verflow or **U**nderflow exception will be detected if the result exceeds the range limits of REAL10 numbers, setting the related flag in the Status Word and the value of INFINITY will overwrite the content of the destination register in case of overflow.

A **Z**ero divide exception will be detected if the value of the source operand is zero. It would yield a signed INFINITY result (unless the value of the destination is also zero which would be an invalid operation).

If a REAL4 or REAL8 value in memory is specified as the source, it is first converted to the REAL10 format before the subtraction. If that value is not an exact number in binary, the precision of the result will only be equivalent to the precision of the source.

Examples of use:

```
fdiv real8_var       ;divide ST(0) by the value of the real8_var variable
fdiv dword ptr [eax] ;divide ST(0) by the REAL4 value pointed to by EAX
fdiv st,st           ;would result in a value of 1.0 in ST(0)
fdiv st,st(3)        ;divide ST(0) by the value of ST(3), result in ST(0)
fdiv st(3),st        ;divide ST(3) by the value of ST(0), result in ST(3)
```

FDIVP (Divide two floating point values and pop ST(0))

```
Syntax:    fdivp st(i),st

Exception flags: Stack Fault, Invalid operation, Denormalized value,
                 Overflow, Underflow, Precision, Zero divide
```

This instruction performs a signed division of the content of the ST(i) data register by the content of the ST(0) data register. The result then overwrites the content of the ST(i) register and the TOP data register is popped.

An **I**nvalid operation exception is detected if a specified data register is empty, if either of the two values is a NAN, if the content of the ST(i) register is INFINITY, or if both values are zero, setting the related flag in the Status Word. The ST(i) register would be overwritten with the INDEFINITE value. (A value of INFINITY will be treated as a valid number for ST(0) and yield a signed zero result without any exception being detected, unless both values are INFINITY regardless of sign).

A **S**tack **F**ault exception is also detected if a data register specified as an operand is empty, setting the related flag in the Status Word.

A **D**enormal exception is detected when either of the two operands is a denormalized number, setting the related flag in the Status Word. The division would still yield a valid result.

A **P**recision exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An **O**verflow or **U**nderflow exception will be detected if the result exceeds the range limits of REAL10 numbers, setting the related flag in the Status Word and the value of INFINITY will overwrite the content of the destination register in case of overflow.

A **Z**ero divide exception will be detected if the value of ST(0) is zero. It would yield a signed INFINITY result (unless the value of ST(i) is also zero which would be an invalid operation).

This instruction is used when the value in ST(0) would no longer be needed for further computation. An example of use could be when a division of ST(0) by a temporary REAL10 value located in memory is needed, which requires loading it to the FPU for such an operation.

```
fld   real10_var ;load the temporary value
                 ;-> ST(0)=real10_var, ST(1)=original ST(0)
fdivp st(1),st   ;divide the original ST(0) by real10_var and discard it
                 ;-> ST(0)=(original ST(0) ï¿½ temporary value)
```

Note that MASM would also accept **fdiv** (no operand) instead of the **fdivp st(1),st** instruction.

FDIVR (Reverse divide two floating point values)

```
Syntax:    fdivr Src
           fdivr Dest,Src
```

Note: FDIVR without operands can also be used with the MASM assembler but such an instruction is coded as FDIVRP ST(1),ST(0).

```
Exception flags: Stack Fault, Invalid operation, Denormalized value,
                 Overflow, Underflow, Precision, Zero divide
```

This instruction performs a signed division of the content of the source (*Src*) operand by the content of the destination (*Dest*) register operand and overwrites the content of the destination data register with the result; the value of the source remains unchanged. If only the source is specified, the TOP data register ST(0) is the implied destination and the source must be the memory address of a REAL4 or REAL8 value (see Chap.2 for addressing modes of real numbers). If both the source and destination are specified, they must both be FPU data registers and one of them must be the TOP data register ST(0). *(The FIDIVR instruction must be used to divide the value of an integer located in memory by ST(0)).*

> Note that a REAL10 in memory cannot be used directly as the source operand. If such a division becomes necessary, the REAL10 value must first be loaded to the FPU.

An **I**nvalid operation exception is detected if a specified data register is empty, if either of the two values is a NAN, if the content of the source operand is INFINITY, or if both values are zero, setting the related flag in the Status Word. The destination register would be overwritten with the INDEFINITE value. (A value of INFINITY will be treated as a valid number for the destination register and yield a signed zero result without any exception being detected, unless both values are INFINITY regardless of sign).

A **S**tack **F**ault exception is also detected if a data register specified as an operand is empty, setting the related flag in the Status Word.

A **D**enormal exception is detected when either of the two operands is a denormalized number, setting the related flag in the Status Word. The division would still yield a valid result.

A **P**recision exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An **O**verflow or **U**nderflow exception will be detected if the result exceeds the range limits of REAL10 numbers, setting the related flag in the Status Word and the value of INFINITY will overwrite the content of the destination register in case of overflow.

A **Z**ero divide exception will be detected if the value of the destination register is zero. It would yield a signed INFINITY result (unless the value of the source operand is also zero which would be an invalid operation).

If a REAL4 or REAL8 value in memory is specified as the source, it is first converted to the REAL10 format before the subtraction. If that value is not an exact number in binary, the precision of the result will only be equivalent to the precision of the source.

Examples of use:

```
fdivr real8_var       ;divide the value of the real8_var variable by ST(0)
                      ;and store the result in ST(0)
fdivr dword ptr [eax] ;divide the REAL4 value pointed to by EAX by ST(0)
                      ;and store the result in ST(0)
fdivr st,st           ;would result in a value of 1.0 in ST(0)
fdivr st,st(3)        ;divide ST(3) by the value of ST(0), result in ST(0)
                      ;ST(3) remains unchanged
fdivr st(3),st        ;divide ST(0) by the value of ST(3), result in ST(3)
                      ;ST(0) remains unchanged
```

FDIVRP (Reverse divide two floating point values and pop ST(0))

```
Syntax:    fdivrp st(i),st

Exception flags: Stack Fault, Invalid operation, Denormalized value,
                 Overflow, Underflow, Precision, Zero divide
```

This instruction performs a signed division of the content of the ST(0) data register by the content of the ST(i) data register. The result then overwrites the content of the ST(i) register and the TOP data register is popped.

An **I**nvalid operation exception is detected if a specified data register is empty, if one of the two values is a NAN, if the content of the ST(0) register is INFINITY, or if both values are zero, setting the related flag in the Status Word. The ST(i) register would be overwritten with the INDEFINITE value. (A value of INFINITY will be treated as a valid number for ST(i) and yield a signed zero result without any exception being detected, unless both values are INFINITY regardless of sign).

A **S**tack **F**ault exception is also detected if a data register specified as an operand is empty, setting the related flag in the Status Word.

A **D**enormal exception is detected when either of the two operands is a denormalized number, setting the related flag in the Status Word. The division would still yield a valid result.

A **P**recision exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

An **O**verflow or **U**nderflow exception will be detected if the result exceeds the range limits of REAL10 numbers, setting the related flag in the Status Word and the value of INFINITY will overwrite the content of the destination register in case of overflow.

A **Z**ero divide exception will be detected if the value of ST(i) is zero. It would yield a signed INFINITY result (unless the value of ST(0) is also zero which would be an invalid operation).

This instruction is used when the value in ST(0) would no longer be needed for further computation. An example of use could be when the content of one of the data registers needs to be replaced by its reciprocal. ST(2) is used for the following code example.

```
fld1              ;load the hard-coded FPU constant 1.0
                  ;-> ST(0)=1, ST(3)=original ST(2)
fdivrp st(3),st   ;divide 1 by the original ST(2) and discard the constant
                  ;-> ST(2)=(1 ï¿½ original ST(2))
```

Note that MASM would also accept **fdivr** (no operand) instead of the **fdivrp st(1),st** instruction.

FABS (Absolute value of ST(0))

```
Syntax:    fabs (No operand)

Exception flags: Stack Fault, Invalid operation
```

This instruction simply clears the sign bit of the value in ST(0), insuring a positive value, i.e. the absolute value.

An **I**nvalid operation exception and a **S**tack **F**ault exception are detected if ST(0) is empty, setting the related flag in the Status Word, and inserting the INDEFINITE NAN into ST(0).

There are numerous reasons for insuring that a value be positive. Examples are when it is necessary to extract its square root or compute its logarithm, which can only be performed on positive values by the FPU.

FCHS (Change the sign of ST(0))

```
Syntax:    fchs (No operand)

Exception flags: Stack Fault, Invalid operation
```

This instruction simply reverses the sign bit of the value in ST(0), changing it from positive to negative or vice versa.

An **I**nvalid operation exception and a **S**tack **F**ault exception are detected if ST(0) is empty, setting the related flags in the Status Word, and inserting the INDEFINITE NAN into ST(0).

There are numerous reasons for changing the sign of a value. For examples, changing the sign of the logarithm of a number effectively yields the logarithm of the reciprocal of that number.

If the sign of the value in a register other than ST(0) is required, it is a simple matter to exchange the contents of that register and of ST(0), change the sign, and exchange once more. The following code changes the sign of the value in ST(2)

```
fxch st(2)  ;exchange the contents of ST(0) and ST(2)
            ;-> ST(0)=original ST(2), ST(2)=original ST(0)
fchs        ;change the sign of ST(0) which is the original ST(2)
fxch st(2)  ;exchange the contents of ST(0) and ST(2)
            ;-> ST(0)=original ST(0), ST(2)=original ST(2) with sign changed
```

If the sign of the value in ST(1) needed to be changed, the following code could also be used. However, any advantage would be very minimal, code size being the same and speed gain possibly only a fraction of a nanosecond.

```
fincstp   ;increment the TOP data register field in the Status Word
          ;ST(0)=>ST(7), ST(1)=>ST(0)
fchs      ;change the sign of ST(0) (formerly the ST(1) value)
fdecstp   ;decrement the TOP data register in the Status Word
          ;ST(7)=>ST(0), ST(0)=>ST(1)
          ;-> ST(1)=original ST(1) with sign changed
```

FSQRT (Square root of ST(0))

```
Syntax:    fsqrt (No operand)

Exception flags: Stack Fault, Invalid operation, Denormalized value,
                 Precision
```

This instruction extracts the square root of the value in ST(0) and overwrites the content of ST(0) with the result.

An **I**nvalid operation exception is detected if ST(0) is empty or contains a negative value, setting the related flag in the Status Word, and overwriting the content of ST(0) with the INDEFINITE NAN.

A **S**tack **F**ault exception is also detected if ST(0) is empty, setting the related flag in the Status Word.

A **D**enormal exception is detected if the content of ST(0) is a denormalized number, setting the related flag in the Status Word.

A **P**recision exception will be detected if some fraction bit is lost due to rounding, setting the related flag in the Status Word.

The values of NANs, +INFINITY and -0 remain unchanged without any exception being detected.

FRNDINT (Round ST(0) to an integer)

```
Syntax:    frndint (No operand)

Exception flags: Stack Fault, Invalid operation, Denormalized value,
                 Precision
```

This instruction rounds the value of ST(0) to the nearest integral value according to the rounding mode of the RC field in the Control Word and overwrites the content of ST(0) with the result.

An **I**nvalid operation exception and a **S**tack **F**ault exception are detected if ST(0) is empty, setting the related flag in the Status Word, and inserting the INDEFINITE NAN into ST(0).

A **D**enormal exception is detected if the content of ST(0) is a denormalized number, setting the related flag in the Status Word.

A **P**recision exception will be detected if the content of ST(0) is not an integral value, setting the related flag in the Status Word.

Values of INFINITY and NANs remain unchanged without any exception being detected.

Although a similar result could be obtained by storing to memory the value of ST(0) as an integer, it would have to be loaded back to the FPU if that rounded value is required for further computation. The FRNDINT is much faster and more efficient. The size of the value is also immaterial with this instruction while the range of an integer variable could possibly be exceeded.

This instruction is also useful prior to storing a value in the packed BCD format with the FBSTP instruction if a different rounding mode must be used.

RETURN TO
SIMPLY FPU
CONTENTS